SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 2

Error Analysis Task

COURSE NAME: Deep Learning COURSE CODE: MLA0403 FACULTY : Dr.Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 30%

Duration: 30 minutes

Marks: 25

Code of Conduct:

I A Afran, 192325024 (Name/Reg.No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Name of the student: A Afran

QUESTIONS: Total Marks:25

1. Error Analysis in Maximum Likelihood Estimation (MLE) A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.
b) Explain how this mistake affects parameter estimation.
c) Suggest the correct mathematical formulation and reasoning.
d) How would using log-likelihood fix computational issues?

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.
b) Analyze how incorrect learning rate or weight updates affect convergence.
c) Identify dataset-related issues that may cause this problem.
d) Propose modifications to ensure convergence.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.
b) Explain the role of activation functions in gradient flow.
c) Analyze how vanishing or exploding gradients affect training.
d) Suggest techniques to fix the issue (e.g., normalization, initialization).

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.
b) Analyze the cause of fluctuations in SGD.
c) Compare the error behavior of Mini-Batch Gradient Descent.
d) Recommend the best approach and justify with error trade-offs.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality. **Tasks:**

a) Explain how high dimensionality increases model error.
b) Identify signs of overfitting in this scenario.
c) Analyze why distance-based learning fails in high dimensions.
d) Suggest dimensionality reduction or regularization techniques to improve performance.

RUBRICS FOR CALCULATION:

Criteria	Weightag e	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem Solving Approach	20	Logical, well structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

ANSWERS

Name: A Afran Reg. No: 192325024 Course: Deep Learning (MLA0403)

1. Error Analysis in Maximum Likelihood Estimation (MLE)

a) Conceptual error:

The student wrote the likelihood as a sum of individual probabilities instead of their product. Since the data points are assumed independent and identically distributed (i.i.d.), the joint likelihood of observing the entire dataset must be the product of each observation's individual probability, not their sum. Adding probabilities does not represent the joint probability of independent events.

b) Effect on parameter estimation:

Using a sum breaks the statistical meaning of the objective function. It no longer represents the true joint probability of the observed data under the model, so maximizing it does not correspond to maximizing the actual likelihood. As a result, the optimizer converges to parameter values that do not best explain the data. This is why the model produces poor estimates even as the dataset size grows -- more data reinforces the wrong objective rather than correcting it.

c) Correct formulation:

For i.i.d. samples (x1,y1), (x2,y2), ..., (xn,yn), the likelihood function should be defined

as the product: L(theta) = P(y1,y2,...,yn | x1,x2,...,xn ; theta) = the product over i = 1 to n of P(yi | xi ; theta). The parameters theta are then estimated as theta_MLE = argmax over theta of L(theta), i.e. the values that make the observed data most probable under the model.

d) Role of log-likelihood:

Taking the logarithm converts the product into a sum: log L(theta) = the sum over i = 1 to n of log P(yi | xi ; theta). Since the logarithm is a strictly increasing function, maximizing the log-likelihood gives the same optimal parameters as maximizing the likelihood itself. This fixes two computational issues: (1) multiplying many small probabilities causes numerical underflow, which summing logs avoids, and (2) differentiating a sum of log terms is far simpler than differentiating a large product, which makes gradient-based optimization (gradient ascent/descent on log-likelihood) tractable.

2. Error Diagnosis in Perceptron Learning

a) Possible implementation errors:

Incorrect weight-update rule (e.g., wrong sign, or forgetting to multiply by the input or the error term), the bias term not being updated, weights never being initialized or reset incorrectly each epoch, an incorrect or missing stopping condition, or the predicted output being computed with the wrong activation/threshold.

b) Effect of learning rate and weight updates:

If the learning rate is too large, each update overshoots the correct decision boundary, causing the weights to oscillate back and forth instead of settling, so the model appears to never converge. If the learning rate is too small, updates move in the correct direction but so slowly that convergence looks like it is not happening within the given number of iterations. If the weight-update formula itself has a sign or term error (e.g., using w = w - eta*(y - yhat)*x instead of w = w + eta*(y - yhat)*x), the weights can be pushed away from the correct boundary rather than toward it, preventing convergence altogether.

c) Dataset-related issues:

Even though the data is described as linearly separable, mislabeled points, duplicate points with contradictory labels, or unscaled/unnormalized features (very different ranges across features) can make the effective decision problem behave as if it is not cleanly separable, or can make certain weight directions update far faster than others, disrupting convergence.

d) Proposed modifications:

Verify the update rule matches w = w + eta*(y - yhat)*x and b = b + eta*(y - yhat); use a small, fixed learning rate as the Perceptron Convergence Theorem only guarantees convergence for a learning rate that does not change the separating direction; normalize/standardize input features; check the dataset for mislabeled or duplicate points; and cap the number of epochs while checking that the weight vector stabilizes rather than oscillates.

3. Backpropagation Gradient Error Analysis

a) Possible mathematical/coding errors:

The most common cause of loss increasing instead of decreasing is an incorrect sign in the gradient descent update, i.e. updating weights as w = w + eta * gradient instead of w = w - eta * gradient, which pushes the weights in the direction that increases the loss. Other causes include an incorrectly applied chain rule (missing a derivative term of the activation function during backpropagation), an incorrect derivative of the loss function, mismatched matrix/vector dimensions causing wrong values to propagate, or a learning rate that is far too high.

b) Role of activation functions in gradient flow:

During backpropagation, the gradient at each layer is multiplied by the derivative of that layer's activation function. Functions such as sigmoid and tanh have derivatives bounded within a small range (sigmoid's maximum derivative is only 0.25), so as the gradient is propagated backward through many layers, it is repeatedly multiplied by these small values and shrinks rapidly. ReLU avoids this for positive inputs since its derivative is 1, but can produce zero gradients ("dying ReLU") for negative inputs.

c) Vanishing vs exploding gradients:

Vanishing gradients occur when repeated multiplication of small derivatives across many layers causes the gradient to shrink toward zero, so the early layers of the network receive almost no learning signal and training stalls with a flat, non-decreasing loss. Exploding gradients occur when repeated multiplication of large derivatives or large weight values causes the gradient to grow exponentially across layers; this produces very large weight updates, which is exactly what would cause the loss to increase or diverge (or become NaN) instead of decreasing.

d) Techniques to fix the issue:

Correct the gradient descent update sign; apply gradient clipping to control exploding gradients; replace sigmoid/tanh with ReLU or Leaky ReLU in hidden layers to improve gradient flow; use proper weight initialization such as Xavier (for tanh/sigmoid) or He initialization (for ReLU); add batch normalization to stabilize the distribution of activations between layers; and reduce the learning rate so updates do not overshoot.

4. Gradient Descent vs SGD Error Investigation

a) Why batch gradient descent is slow on large datasets:

Batch gradient descent computes the gradient of the loss using the entire training dataset before performing a single weight update. For a large dataset with n samples, this means every single iteration requires a full pass over all n samples, making each update computationally expensive; convergence, while stable, requires many such expensive passes, so total training time becomes very slow as n grows.

b) Cause of fluctuations in SGD:

SGD updates the weights using the gradient computed from just one (or a very small number of) training samples at a time. This gradient is a noisy, high-variance estimate of the true gradient over the whole dataset, since a single sample may not represent the overall data distribution. As a result, each update can move the weights in a slightly different direction, causing the loss to fluctuate up and down rather than decreasing smoothly, even though it trends downward on average over time.

c) Error behaviour of Mini-Batch Gradient Descent:

Mini-batch gradient descent computes the gradient over a small batch of samples (e.g. 32-256) rather than one sample or the entire dataset. Averaging the gradient over a batch reduces the variance/noise compared to pure SGD, giving smoother, more stable convergence, while still being far more computationally efficient per update than full batch gradient descent. Its error behaviour therefore sits between the two: less noisy than SGD, but faster per update than batch GD.

d) Recommended approach:

Mini-batch gradient descent is generally the best trade-off: it retains enough of SGD's frequent, efficient updates and noise (which can help escape shallow local minima) while averaging over a batch to reduce the fluctuation seen in pure SGD, and it makes better use of vectorized computation than processing one sample at a time. Combining this with a learning rate schedule (decaying the learning rate over time) further reduces late-stage fluctuations while keeping early training fast.

5. Curse of Dimensionality Error Analysis

a) How high dimensionality increases model error:

As the number of features/dimensions increases, the volume of the feature space grows exponentially, so a fixed amount of training data becomes increasingly sparse within that space. With sparse data, a flexible model can fit the training points almost perfectly, including their noise, because there are effectively very few real constraints per added dimension. This lowers training error but increases variance, so the gap between training and test error (generalization error) grows -- this is overfitting driven by high dimensionality.

b) Signs of overfitting in this scenario:

Very low or near-zero training error combined with noticeably higher test error; a decision boundary or function that is unnecessarily complex/wiggly; performance that degrades further as more irrelevant features are added; and predictions that are highly sensitive to small changes in the test data are all signs consistent with the overfitting described.

c) Why distance-based learning fails in high dimensions:

Distance-based methods (e.g. k-Nearest Neighbours) rely on the assumption that points which are close together are meaningfully similar. In high-dimensional space, it can be shown that the ratio between the distance to the nearest neighbour and the distance to the farthest neighbour approaches 1 as dimensionality grows -- i.e., all points appear roughly equidistant from each other. This causes the concept of a 'neighbourhood' to lose its discriminative meaning, so distance-based predictions become unreliable.

d) Suggested techniques:

Apply dimensionality reduction methods such as PCA, t-SNE, or autoencoders to project data onto a lower-dimensional space that retains the most informative variance; perform feature selection to remove irrelevant or redundant features; apply regularization (L1/L2) to constrain model complexity; and, where possible, collect more training data or use

domain knowledge to restrict the feature set to genuinely relevant variables.